

# Assignment 2

## Single-Layer Perceptron for Binary Classification

Deep Learning Practical | CSE-AI | Semester 5 | Academic Year 2026-27

### 1. Aim

Implement a Single-Layer Perceptron for binary classification and analyze the effect of weights, bias, and activation functions.

### 2. Problem Statement

Implement a Single-Layer Perceptron and study how the learned weights, bias, and different activation functions influence binary classification.

### 3. Dataset

The notebook uses the OR-gate truth table with two input features and a binary target.

| X1 | X2 | Target |
|----|----|--------|
| 0  | 0  | 0      |
| 0  | 1  | 1      |
| 1  | 0  | 1      |
| 1  | 1  | 1      |

### 4. Algorithm

1. Initialize weights and bias. 2. Compute  $z = X \cdot W + b$ . 3. Apply the step function. 4. Compute error = target – prediction. 5. Update weights and bias using the perceptron learning rule. 6. Repeat for 10 epochs. 7. Test all input combinations. 8. Compare activation functions.

### 5. Implementation Code

```
import numpy as np

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 1])

def step_function(x):
    return 1 if x >= 0 else 0

weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10

for epoch in range(epochs):
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        prediction = step_function(linear_output)
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error
```

### 6. Training Result

Final weights = [0.1, 0.1]

Final bias = -0.1

Predictions = [0, 1, 1, 1]

Actual = [0, 1, 1, 1]

Training accuracy = 100.00%

## 7. Analysis of Weights and Bias

Weights determine how strongly each input contributes to the linear decision value. The bias shifts the decision boundary. Changing either can change which side of the threshold an input falls on. The notebook explicitly tests multiple weight/bias combinations to demonstrate this effect.

## 8. Activation Function Analysis

The Step function produces a binary output and is the activation used for the perceptron decision. Sigmoid converts the linear output into a value between 0 and 1, while ReLU outputs zero for negative values and the positive input otherwise. The notebook prints all three outputs for each OR-gate sample.

## 9. Important Source Note

The uploaded practical sheet describes the target as an AND gate, but the uploaded notebook implements an OR gate with targets [0, 1, 1, 1]. This report follows the notebook's actual implementation. If the practical must strictly be AND, use targets [0, 0, 0, 1].

## 10. Conclusion

The Single-Layer Perceptron successfully learns the OR-gate binary classification task. The experiment demonstrates how weights and bias control the decision boundary and how different activation functions transform the linear output.

## 11. Full Code

The complete executable notebook is included as `single_layer_perceptron.ipynb` in this assignment folder.